

GitHub Task: Lexical Analyzer & Token Counter

Objective

Develop a program that reads a source-code file and performs lexical analysis by identifying and counting different types of tokens.

Problem Statement

Design and implement a Lexical Analyzer in C that reads a C statement as input and identifies the different types of tokens present in it. The program should classify each token as a keyword, identifier, operator, constant, string literal, or separator, display each token with its type, and finally calculate and display the count of each type of token and the total number of tokens.

Algorithm

Step 1: Start the program.

Step 2: Define a list of C language keywords such as int, float, if, else, return, while, for, etc.

Step 3: Read the source code input using fgets().

Step 4: Scan the input character by character until the end of the input.

Step 5: If the character is a whitespace, ignore it and move to the next character.

Step 6: If the character is an alphabet or underscore (_):

- a. Read all following characters that are alphabets, digits, or underscores.
- b. Form a complete word.
- c. Check whether the word exists in the keyword list.
- d. If it is a keyword, classify it as Keyword.
- e. Otherwise, classify it as Identifier.
- f. Increment the corresponding counter and total token count.

Step 7: If the character is a digit:

- a. Read the complete numeric value.
- b. Classify it as a Constant.
- c. Increment the constant counter and total token count

Step 8: If the character is a double quote ("):

- a. Read characters until the closing double quote is found.
- b. Classify the complete sequence as a String Literal.
- c. Increment the string literal counter and total token count.

Step 9: If the character is an operator (+, -, *, /, =, <, >, etc.):

- a. Check whether it forms a two-character operator such as ==, <=, >=, or !=.
- b. Classify it as an Operator.
- c. Increment the operator counter and total token count.

Step 10: If the character is a separator such as ;, ,, (,), {, }, [,]:

- a. Classify it as a Separator.
- b. Increment the separator counter and total token count.

Step 11: Ignore any unknown or unsupported characters.

Step 12: Display each identified token along with its token type.

Step 13: Display the final token count:

- Keywords
- Identifiers
- Operators
- Constants
- String Literals
- Separators
- Total Tokens

Step 14: Stop the program.

Source Code

Save the following program as lexical_analyzer.c.

```
#include <stdio.h>

#include <string.h>

#include <ctype.h>

int isKeyword(char str[])
```

```
{
    char keywords[32][10] = {
        "auto", "break", "case", "char", "const", "continue",
        "default", "do", "double", "else", "enum", "extern",
        "float", "for", "goto", "if", "int", "long",
        "register", "return", "short", "signed", "sizeof",
        "static", "struct", "switch", "typedef", "union",
        "unsigned", "void", "volatile", "while"
    };
    int i;
    for (i = 0; i < 32; i++)
    {
        if (strcmp(str, keywords[i]) == 0)
            return 1;
    }
    return 0;
}

int main()
{
    FILE *fp;
    char ch, token[100];
    int i, j;
    int keywords = 0;
    int identifiers = 0;
    int operators = 0;
    int constants = 0;
    int strings = 0;
    int separators = 0;
    int comments = 0;
```

```
fp = fopen("sample_input.c", "r");
if (fp == NULL)
{
    printf("Error: Cannot open sample_input.c\n");
    return 1;
}
printf("TOKEN TYPE\n");
printf("-----\n");
while ((ch = fgetc(fp)) != EOF)
{
    if (isspace(ch))
        continue;
    /* Identifier or Keyword */
    if (isalpha(ch) || ch == '_')
    {
        j = 0;
        token[j++] = ch;
        while ((ch = fgetc(fp)) != EOF &&
            (isalnum(ch) || ch == '_'))
        {
            token[j++] = ch;
        }
        token[j] = '\0';
        if (isKeyword(token))
        {
            printf("%s : Keyword\n", token);
            keywords++;
        }
        else
```

```
{
    printf("%s : Identifier\n", token);
    identifiers++;
}
if (ch != EOF)
    ungetc(ch, fp);
}
/* Constant */
else if (isdigit(ch))
{
    j = 0;
    token[j++] = ch;
    while ((ch = fgetc(fp)) != EOF &&
        (isdigit(ch) || ch == '.'))
    {
        token[j++] = ch;
    }
    token[j] = '\0';
    printf("%s : Constant\n", token);
    constants++;
    if (ch != EOF)
        ungetc(ch, fp);
}
/* String Literal */
else if (ch == '"')
{
    j = 0;
    token[j++] = ch;
    while ((ch = fgetc(fp)) != EOF && ch != '"')
```

```
{
    token[j++] = ch;
}
if (ch == '"')
    token[j++] = ch;
token[j] = '\0';
printf("%s : String Literal\n", token);
strings++;
}
/* Comments or Division Operator */
else if (ch == '/')
{
    char next = fgetc(fp);
    if (next == '/')
    {
        printf("// : Comment\n");
        comments++;
        while ((ch = fgetc(fp)) != EOF && ch != '\n');
    }
    else if (next == '*')
    {
        printf("/* */ : Comment\n");
        comments++;
        while ((ch = fgetc(fp)) != EOF)
        {
            if (ch == '*' && (ch = fgetc(fp)) == '/')
                break;
        }
    }
}
```

```
    else
    {
        printf("/ : Operator\n");
        operators++;
        if (next != EOF)
            ungetc(next, fp);
    }
}

/* Operators */
else if (ch == '+' || ch == '-' ||
        ch == '*' || ch == '=' ||
        ch == '<' || ch == '>' ||
        ch == '%')
{
    printf("%c : Operator\n", ch);
    operators++;
}

/* Separators */
else if (ch == '(' || ch == ')' ||
        ch == '{' || ch == '}' ||
        ch == '[' || ch == ']' ||
        ch == ';' || ch == ',')
{
    printf("%c : Separator\n", ch);
    separators++;
}

/* Other special symbols */
else
{
```

```

        printf("%c : Special Symbol\n", ch);
    }
}
fclose(fp);
printf("\nTOKEN COUNT\n");
printf("-----\n");
printf("Keywords      : %d\n", keywords);
printf("Identifiers   : %d\n", identifiers);
printf("Operators     : %d\n", operators);
printf("Constants     : %d\n", constants);
printf("String Literals: %d\n", strings);
printf("Separators    : %d\n", separators);
printf("Comments     : %d\n", comments);
return 0;
}

```

Sample Input

Create a file named sample_input.c:

```

int sum = a + b;
float average = sum / 2.0;
if (average > 50)
printf("Pass");

```

Sample Output

Enter source-code filename: sample_input.c

TOKEN TYPE

int : Keyword

sum : Identifier

= : Operator

a : Identifier

+ : Operator

b : Identifier

; : Separator

float : Keyword

average : Identifier

= : Operator

sum : Identifier

/ : Operator

2 : Constant

; : Separator

if : Keyword

(: Separator

average : Identifier

> : Operator

50 : Constant

) : Separator

printf : Identifier

(: Separator

"Pass" : String Literal

) : Separator

; : Separator

TOKEN COUNT

Keywords : 3

Identifiers : 7

Operators : 5

Constants : 2

String Literals: 1

Separators : 7

Comments : 0

Token Classification

Token Classification

int Keyword

float Keyword

if Keyword

sum Identifier

average Identifier

a, b Identifier

= Operator

+ Operator

/ Operator

> Operator

2.0, 50 Constant

"Pass" String Literal

(,), ; Separator

// Calculate average Comment

Token Categories

Keywords: Reserved words of the C language such as int, float, and if.

Identifiers: Names given to variables, functions, arrays, etc.

Operators: Symbols that perform operations, such as +, -, =, /, and >.

Constants: Fixed numeric or character values.

String Literals: Text enclosed within double quotation marks.

Separators: Symbols used to separate or group program elements, such as (,), ,, and {.

Special Symbols: Other symbols that do not belong to the above categories.

Comments: Text beginning with // or enclosed between /* and */.

Test Cases

Test Case 1: Basic Arithmetic

Input:

```
int a = 10;
```

```
int b = 20;
```

```
int sum = a + b;
```

Expected classification:

Keywords : 3

Identifiers : 5

Operators : 3

Constants : 2

Separators : 6

Test Case 2: Conditional Statement

Input:

```
if (x > 10)
    printf("Greater");
```

Expected token types:

if -> Keyword

x -> Identifier

> -> Operator

10 -> Constant

printf -> Identifier

"Greater" -> String Literal

() -> Separators

Conclusion

The lexical analyzer successfully reads a C source-code file and identifies different types of tokens. It classifies keywords, identifiers, operators, constants, string literals, separators, special symbols, and comments. It also maintains individual counts for each token category.